Simulação do Brasileirão 2026

Caio Gomes Alves *Instituto de Matemática, Estatística e Computação Científica - UNICAMP*

Simulação do Campeonato

Construção da tabela

Para a simulação, vejamos primeiro como se comportam os valores de θ e ϕ para cada um dos times:

```
dados_bras <- read.csv("~/Downloads/brasileirao_2026.csv")

# Criando as taxas theta e phi:
taxas_estimadas <- left_join(
    # Taxas dos mandantes:
    (dados_bras %>%
    mutate(time = time_mandante) %>%
    group_by(time) %>%
    summarise(theta_m = mean(gols_mandante, na.rm = T),
              phi_v = mean(gols_visitante, na.rm = T))),
    # Taxas dos visitantes:
    (dados_bras %>%
    mutate(time = time_visitante) %>%
    group_by(time) %>%
    summarise(theta_v = mean(gols_visitante, na.rm = T),
              phi_m = mean(gols_mandante, na.rm = T))),
    # Agrupa por time:
    by = "time")
```

Time	theta_m	phi_v	theta_v	phi_m
Athletico-PR	1.769231	0.7692308	1.1666667	1.2500000
Atlético-MG	1.727273	1.2727273	1.0000000	1.0769231
Bahia	1.538461	1.0000000	1.4166667	1.4166667
Botafogo	1.818182	1.5454545	1.3076923	1.7692308
Bragantino	1.416667	0.9166667	1.0000000	1.1666667
Chapecoense	1.416667	2.2500000	0.6666667	1.8333333
Corinthians	1.000000	0.8461538	1.0833333	1.1666667
Coritiba	1.250000	1.1666667	1.3846154	1.4615385
Cruzeiro	1.615385	1.1538462	1.1666667	1.7500000
Flamengo	2.000000	0.6666667	2.0000000	1.0000000
Fluminense	1.769231	1.0769231	1.3333333	1.5000000
Grêmio	1.750000	1.1666667	0.5000000	1.5000000
Internacional	1.076923	1.0000000	1.0000000	1.5000000
Mirassol	1.307692	1.4615385	0.8333333	1.6666667
Palmeiras	1.750000	0.7500000	1.8461538	0.9230769
Remo	1.416667	1.3333333	1.0000000	2.0000000

Time	theta_m	phi_v	theta_v	phi_m
Santos	1.384615	1.3076923	1.4545455	1.7272727
São Paulo	1.636364	0.9090909	0.8461538	1.3846154
Vasco	1.230769	1.3846154	1.0000000	1.9090909
Vitória	1.333333	0.7500000	0.6153846	2.1538462

Com isso, podemos criar uma função que adiciona os valores das taxas calculadas em cada uma das linhas das rodadas do campeonato, criando um data.frame aumentado, que será usado na simulação:

```
cria_df_taxas <- function(df, taxas) {
    times <- unique(taxas$time)
    df_mandante <- left_join(df, (taxas %>%
                                  select(time_mandante = time,
                                         theta_m, phi_m)), by = "time_mandante") %>%
        select(theta_m, phi_m)
    df_visitante <- left_join(df, (taxas %>%
                                   select(time_visitante = time,
                                          theta_v, phi_v)), by = "time_visitante") %>%
        select(theta_v, phi_v)
    df_completo <- cbind(df, df_mandante, df_visitante)
    return(df_completo)
}

df_completo <- cria_df_taxas(dados_bras, taxas_estimadas)
```

Com o data.frame completo, separaremos em jogos realizados (que não precisam passar por simulação) e jogos a realizar (que terão os placares simulados usando os valores de θ e ϕ calculados anteriormente):

```
jogos_realizados <- df_completo %>% filter(!is.na(gols_mandante))
jogos_futuros <- df_completo %>% filter(is.na(gols_mandante))
```

A função que calcula a pontuação do campeonato deve considerar os casos de vitória/empate/derrota, além de calcular os fatores de desempate (número de vitórias, saldo de gols e quantidade de gols marcados):

```
calcula_tabela <- function(df) {
    df %>%
        mutate(
            # Pontos do mandante:
            pts_m = case_when(gols_mandante > gols_visitante ~ 3,
                              gols_mandante == gols_visitante ~ 1,
                              TRUE ~ 0),
            # Pontos do visitante:
            pts_v = case_when(gols_visitante > gols_mandante ~ 3,
                              gols_mandante == gols_visitante ~ 1,
```

```
                                TRUE ~ 0),
            vit_m = ifelse(gols_mandante > gols_visitante, 1, 0),
            vit_v = ifelse(gols_visitante > gols_mandante, 1, 0)
        ) %>%
        # Junta ambos os casos:
        pivot_longer(cols = c(time_mandante, time_visitante),
                     names_to = "mando", values_to = "time") %>%
        mutate(
            pontos = ifelse(mando == "time_mandante", pts_m, pts_v),
            vitorias = ifelse(mando == "time_mandante", vit_m, vit_v),
            gols_pro = ifelse(mando == "time_mandante", gols_mandante,
                              gols_visitante),
            gols_contra = ifelse(mando == "time_mandante", gols_visitante,
                                 gols_mandante),
            saldo = gols_pro - gols_contra
        ) %>%
        group_by(time) %>%
        # Sumariza os valores necessários para compor o campeonato:
        summarise(across(c(pontos, vitorias, saldo, gols_pro),
                         sum, na.rm = T), .groups = "drop")
}

# Resultado até a 25ª rodada:
tabela_base <- calcula_tabela(jogos_realizados)
```

time	pontos	vitorias	saldo	gols_pro	posicao
Palmeiras	52	15	24	45	1
Flamengo	51	15	29	50	2
Athletico-PR	45	13	12	37	3
Fluminense	42	11	7	39	4
Bahia	40	10	7	37	5
Cruzeiro	39	11	-1	35	6
Coritiba	37	10	0	33	7
Atlético-MG	36	10	4	32	8
Bragantino	35	10	4	29	9
Corinthians	32	8	1	26	10
São Paulo	30	8	1	29	11
Botafogo	30	8	-3	37	12
Vitória	29	8	-13	24	13
Santos	29	7	-2	34	14
Grêmio	28	7	-5	27	15
Mirassol	25	6	-12	27	16
Vasco	25	6	-12	27	17
Internacional	25	5	-5	26	18
Remo	23	5	-12	30	19
Chapecoense	14	2	-24	25	20

Com isso, agora construímos uma função que calcula os jogos restantes usando os dados de `jogos_futuros`, concatena as linhas com os jogos já realizados, e calcula a tabela final do campeonato:

```
simula_rodadas_restantes <- function(...) {
    sim_futuros <- jogos_futuros %>%
        mutate(
            # Poissons uando os valores calculados:
            gols_mandante = rpois(n(), (theta_m + phi_v)/2),
            gols_visitante = rpois(n(), (theta_v + phi_m)/2)
        )
    # Calcula as pontuações dos jogos simulados:
    tabela_simulada <- calcula_tabela(sim_futuros)
    # Junta com a pontuação dos jogos realizados:
    tabela_final <- bind_rows(tabela_base, tabela_simulada) %>%
        group_by(time) %>%
        summarise(across(everything(), sum, na.rm = T), .groups = "drop") %>%
        arrange(desc(pontos), desc(vitorias), desc(saldo), desc(gols_pro)) %>%
        mutate(posicao = row_number())
    # Retorna uma lista com informações para responder a todas as perguntas:
    list(
        campeao = tabela_final$time[1],
        pontos_campeao = tabela_final$pontos[1],
        rebaixados = tabela_final$time[17:20],
        pontos_todos = tabela_final %>% select(time, pontos, posicao)
    )
}
```

Simulação das Rodadas Restantes

Com todas essas funções construídas, podemos realizar as simulações das rodadas faltantes. Como trabalho em um ambiente Linux, o uso paralelizado da função `mclapply`, do pacote `parallel` se dá de maneira direta:

```
# Número de simulações:
n_simulacoes <- 10000
# Seed para reprodutibilidade:
set.seed(5050)
# Simulação usando o mclapply:
resultados <- mclapply(1:n_simulacoes, simula_rodadas_restantes,
                       mc.cores = 10)
```

Perguntas

1. Qual a probabilidade de cada time ser campeão?

Com os resultados das 10^4, podemos contar quantas vezes cada time aparece como campeão, e ranqueá-los:

```
left_join(
    data.frame(Time = unique(jogos_futuros$time_mandante) %>% sort()),
    lapply(resultados, function(x){x$campeao}) %>%
    unlist() %>%
    table() %>%
    as.data.frame() %>%
    mutate(Freq = Freq/n_simulacoes) %>%
    select("Time" = ".","Freq"), by = "Time") %>%
    arrange(desc(Freq)) %>%
    mutate(Freq = case_when(is.na(Freq) ~ 0,
                            .default = Freq),
           Freq = scales::percent(Freq)) %>%
    select(Time, "Prob_Camp" = Freq) %>%
    kable()
```

Time	Prob_Camp
Flamengo	57.92%
Palmeiras	38.76%
Athletico-PR	2.85%
Fluminense	0.30%
Bahia	0.09%
Cruzeiro	0.03%
Atlético-MG	0.02%
Bragantino	0.02%
Coritiba	0.01%
Botafogo	0.00%
Chapecoense	0.00%
Corinthians	0.00%
Grêmio	0.00%
Internacional	0.00%
Mirassol	0.00%
Remo	0.00%
Santos	0.00%
São Paulo	0.00%
Vasco	0.00%
Vitória	0.00%

2. Qual a probabilidade de cada time ser rebaixado?

Semelhante ao caso anterior, a função retorna os 4 times que foram rebaixados em cada simulação, podemos contar quantas vezes cada um aparece e ranqueá-los:

```
left_join(
    data.frame(Time = unique(jogos_futuros$time_mandante) %>% sort()),
    lapply(resultados, function(x){x$rebaixados}) %>%
    unlist() %>%
    table() %>%
    as.data.frame() %>%
    mutate(Freq = Freq/(4 * n_simulacoes)) %>%
    select("Time" = ".","Freq"), by = "Time") %>%
    arrange(desc(Freq)) %>%
    mutate(Freq = case_when(is.na(Freq) ~ 0,
                            .default = Freq),
           Freq = scales::percent(Freq)) %>%
    select(Time, "Prob_Reb" = Freq) %>%
    kable()
```

Time	Prob_Reb
Chapecoense	24.8575%
Remo	18.5350%
Mirassol	15.1250%
Vasco	14.7675%
Internacional	12.2175%
Vitória	4.5425%
Grêmio	4.2050%
Santos	2.7100%
São Paulo	1.2225%
Botafogo	0.9200%
Corinthians	0.6900%
Coritiba	0.1000%
Bragantino	0.0525%
Atlético-MG	0.0300%
Cruzeiro	0.0200%
Bahia	0.0050%
Athletico-PR	0.0000%
Flamengo	0.0000%
Fluminense	0.0000%
Palmeiras	0.0000%

3. Qual a probabilidade do campeonato ser decidido pelos critérios de desempate?

Para que ocorra uma decisão por desempate, é necessário que o primeiro e o segundo colocado tenham a mesma pontuação no final do campeonato. Como a função retorna a pontuação final, podemos calcular a probabilidade utilizando uma função indicadora:

```
((sapply(resultados, function(x) {x$pontos_todos[1, 2] == x$pontos_todos[2, 2]}) %>%
    sum())/n_simulacoes) %>% scales::percent(accuracy = 0.01)
```

```
## [1] "6.34%"
```

4. Qual o valor esperado do número de pontos do campeão?

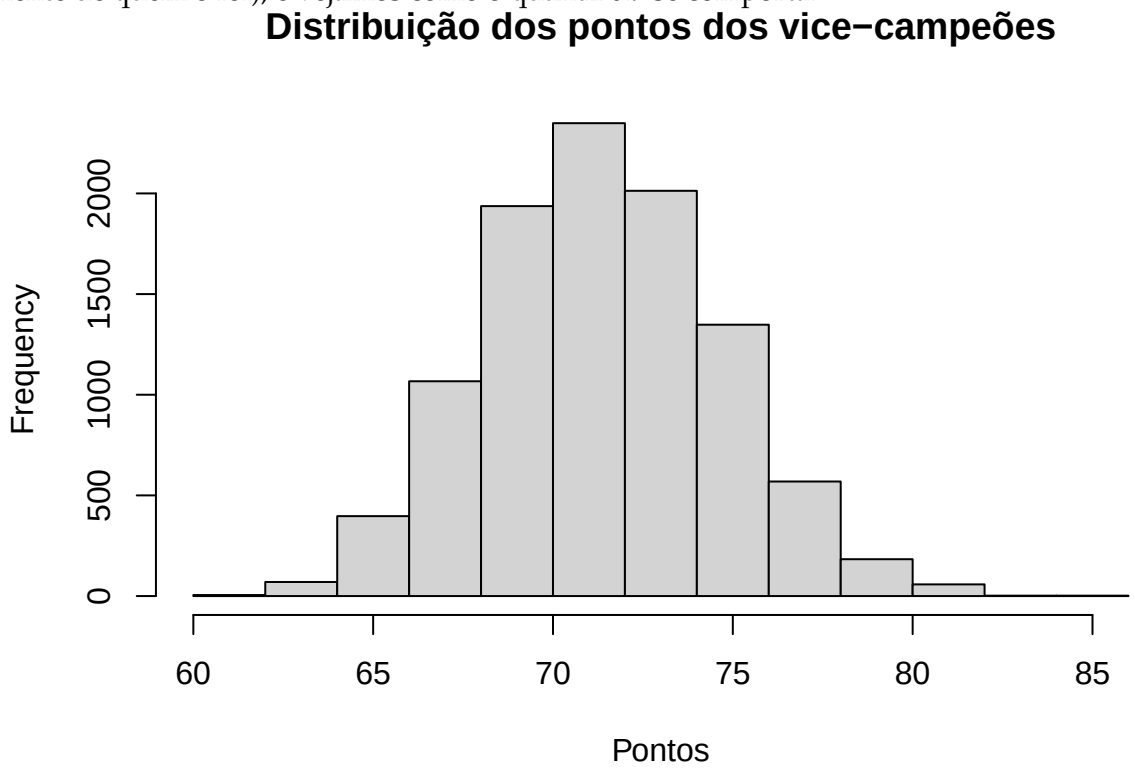
Aqui, consideremos o valor esperado como a média dos pontos obtidos pelo campeão de cada uma das simulações:

```
sapply(resultados, function(x) {x$pontos_campeao}) %>%
    mean()
```

```
## [1] 76.974
```

5. Quantos pontos uma qualquer equipe precisa fazer para ter pelo menos 90% de chance de ser o campeão?

Neste caso, avaliemos a distribuição da pontuação de cada um dos vice-campeões (independentemente de quem o foi), e vejamos como o quantil 0.9 se comporta:



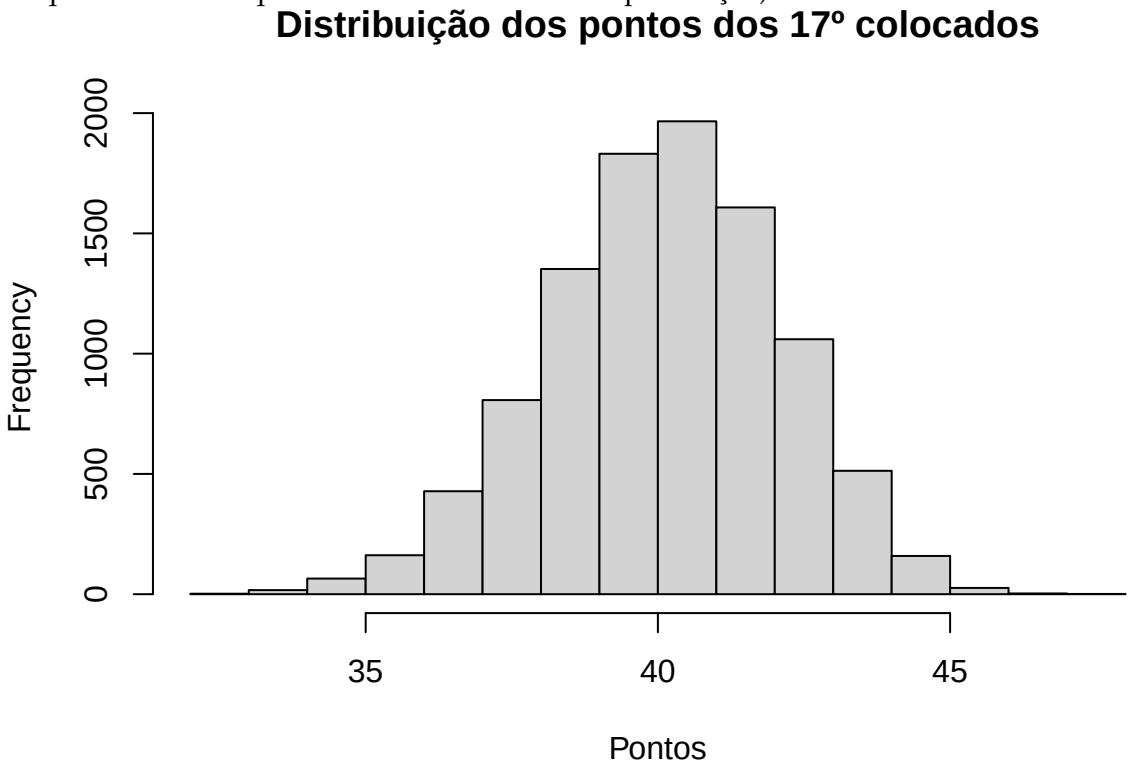
Assim, precisamos extrair o quantil 0.9, usando a função `quantile()`:

```
quantile(lapply(resultados, function(x) {x$pontos_todos[2, 2]}) %>%
         unlist(), 0.9)
```

```
## 90%
##  76
```

6. Quantos pontos são necessários que um time qualquer faça para ter pelo menos 95% de chance de não ser rebaixado?

Semelhante ao caso anterior, vejamos como se comporta a pontuação do 17º colocado na tabela (visto que é ele o time que foi rebaixado com maior pontuação):



Assim, precisamos extrair o quantil 0.95, usando a função `quantile()`:

```
quantile(lapply(resultados, function(x) {x$pontos_todos[17, 2]}) %>%
    unlist(), 0.95)
```

```
## 95%
##  44
```

Bootstrap

Se quisermos avaliar a variabilidade inerente para os θ's e ϕ's, podemos fazer um estudo de reamostragem por bootstrap, e recalcular esses valores uma quantidade suficientemente grande de vezes.

```
# Número de amostras bootstrap:
B <- 1000

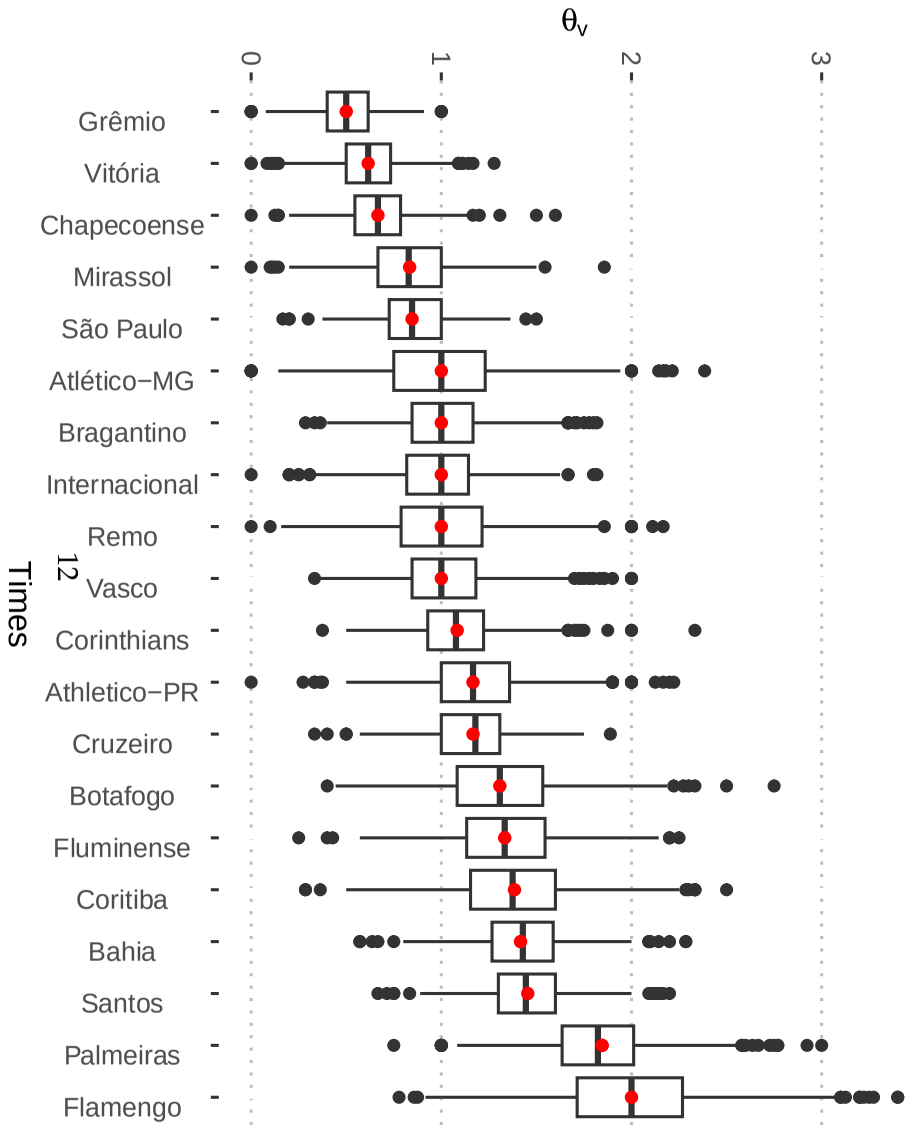
# Matrizes com os valores de cada estimativa:
boot_theta_m <- matrix(NA, nrow = B,
                       ncol = length(unique(jogos_realizados$time_mandante)),
                       dimnames = list(NULL,
                                       unique(jogos_realizados$time_mandante) %>% sort()))
boot_theta_v <- matrix(NA, nrow = B,
                       ncol = length(unique(jogos_realizados$time_mandante)),
                       dimnames = list(NULL,
                                       unique(jogos_realizados$time_mandante) %>% sort()))
boot_phi_m <- matrix(NA, nrow = B,
                     ncol = length(unique(jogos_realizados$time_mandante)),
                     dimnames = list(NULL,
                                     unique(jogos_realizados$time_mandante) %>% sort()))
boot_phi_v <- matrix(NA, nrow = B,
                     ncol = length(unique(jogos_realizados$time_mandante)),
                     dimnames = list(NULL,
                                     unique(jogos_realizados$time_mandante) %>% sort()))

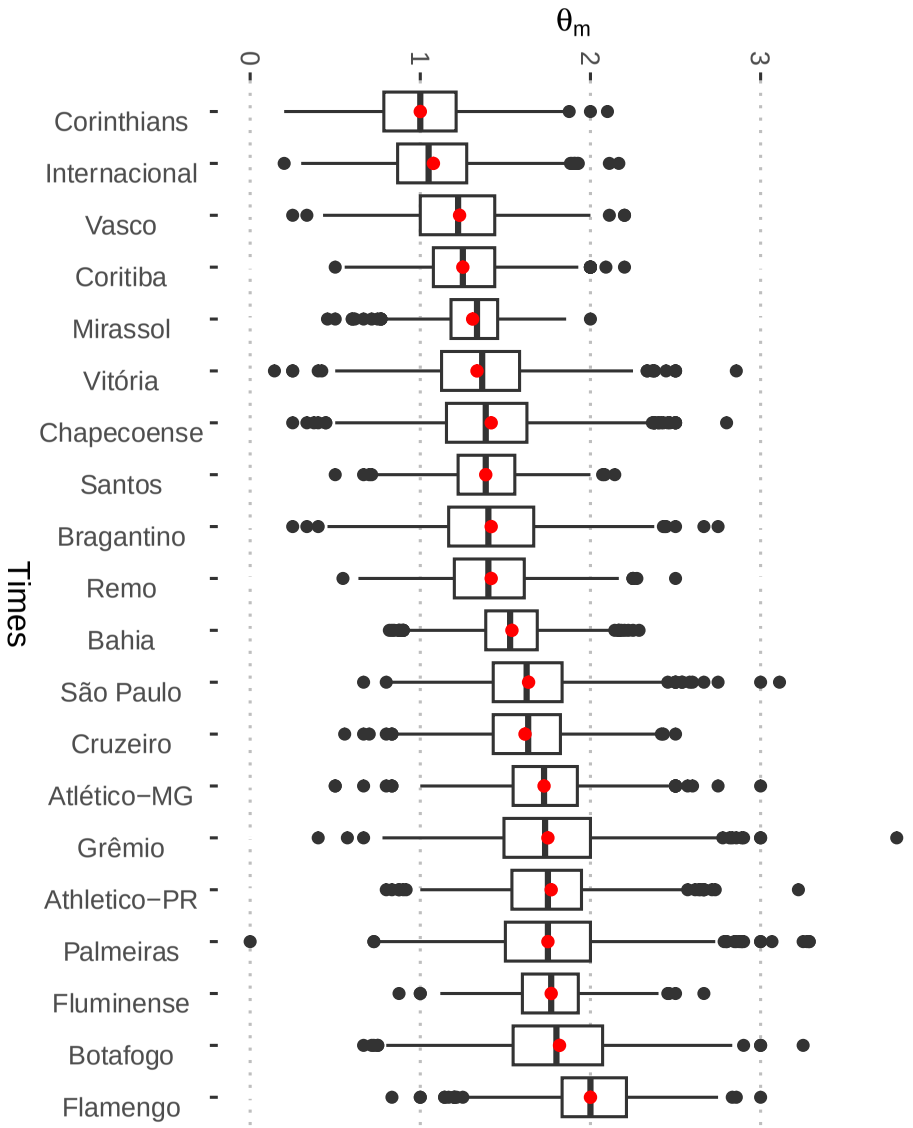
# Seed para reprodutibilidade:
set.seed(5050)

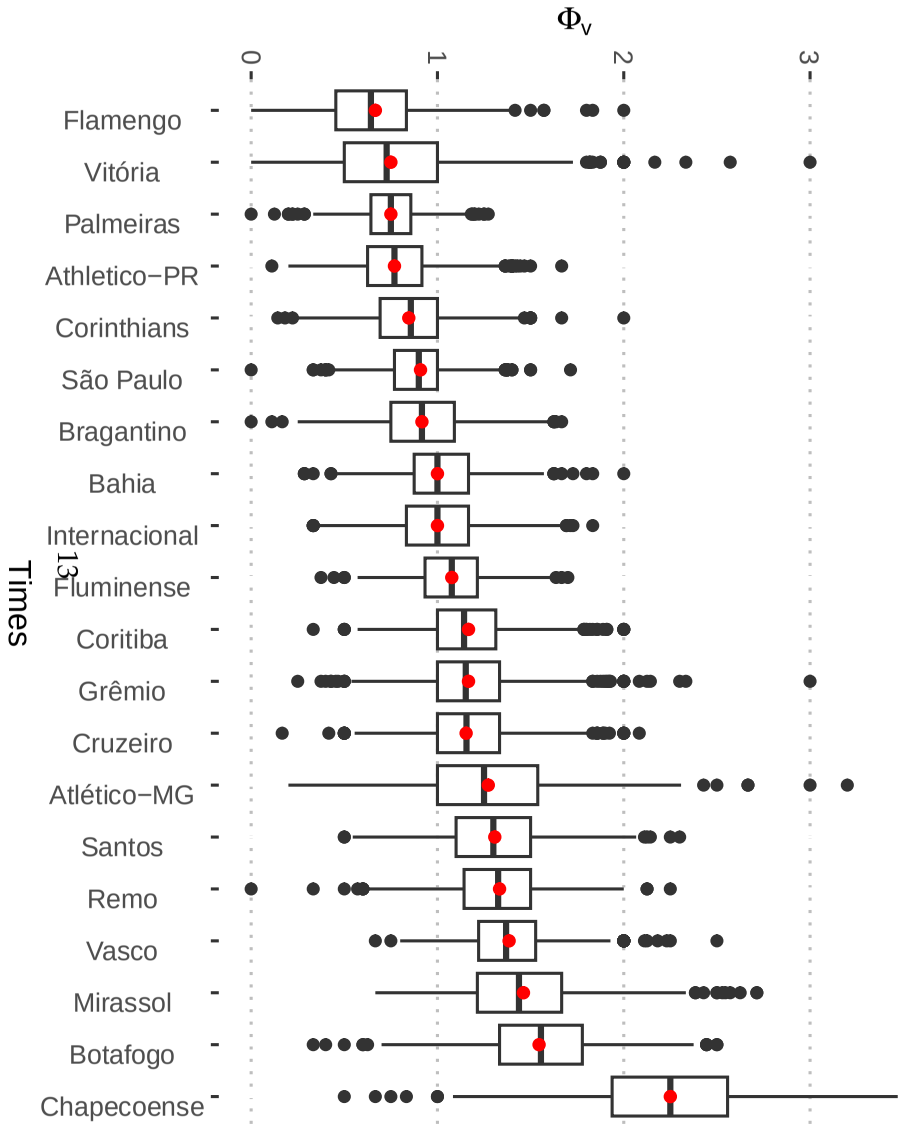
# Bootstrap
for(i in 1:B) {
    # Extrai uma amostra com reposição dos jogos realizados:
    amostra <- jogos_realizados[sample(1:nrow(jogos_realizados),
                                       replace = TRUE), ]
    # Recalcula as taxas:
    taxas_estimadas_boot <- left_join(
    (amostra %>%
     mutate(time = time_mandante) %>%
     group_by(time) %>%
     summarise(theta_m = mean(gols_mandante, na.rm = T),
               phi_v = mean(gols_visitante, na.rm = T))),
    (amostra %>%
     mutate(time = time_visitante) %>%
     group_by(time) %>%
     summarise(theta_v = mean(gols_visitante, na.rm = T),
               phi_m = mean(gols_mandante, na.rm = T))), by = "time")
    # Aplica os valores por linha nas matrizes:
```

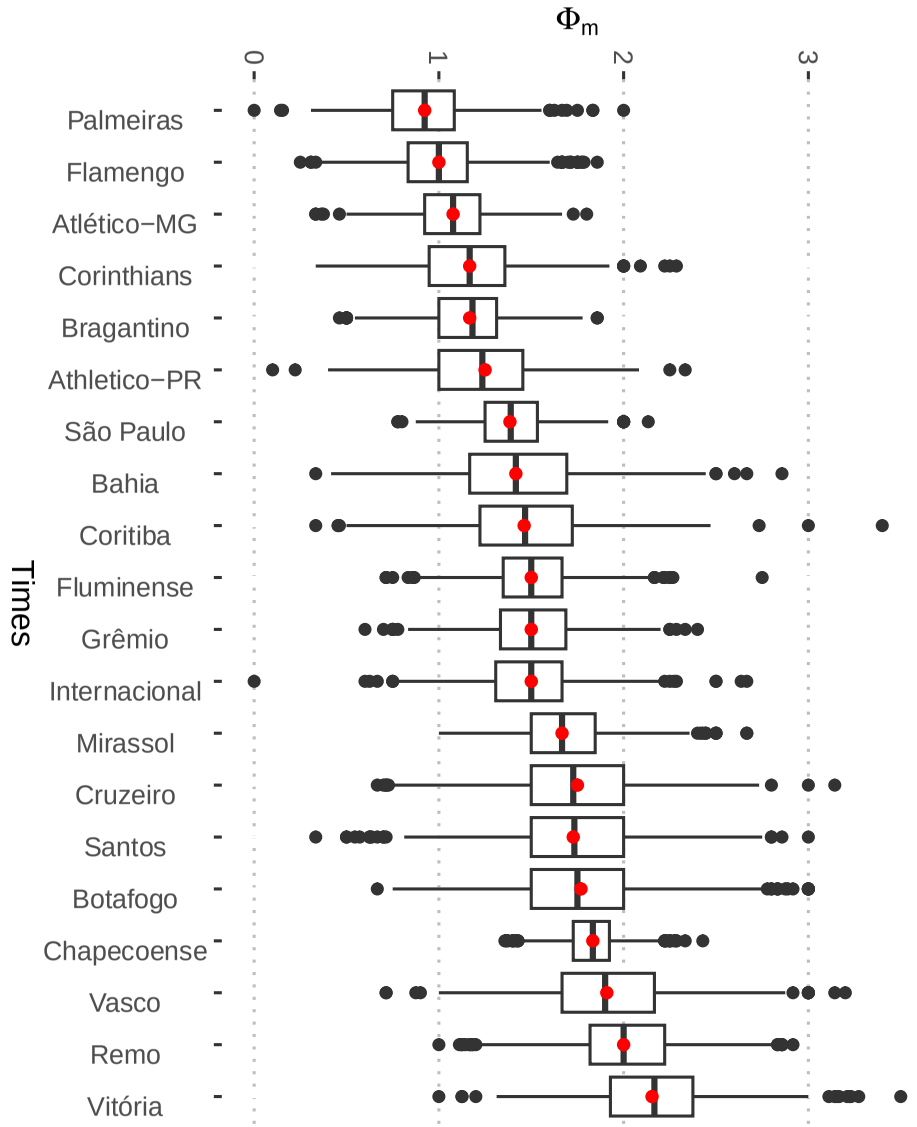
```
    boot_theta_m[i, ] <- taxas_estimadas_boot$theta_m
    boot_theta_v[i, ] <- taxas_estimadas_boot$theta_v
    boot_phi_m[i, ] <- taxas_estimadas_boot$phi_m
    boot_phi_v[i, ] <- taxas_estimadas_boot$phi_v
}
```

Com isso, podemos construir boxplots para cada uma das taxas estimadas, considerando cada time. Neles, teremos ideia da variabilidade por meio do intervalo interquantílico e mediana das reamostras bootstrap. Além disso, o valor original será colocado como um ponto vermelho, para título de comparação.


θv
0
1
2
3
Grêmio
Vitória
Chapecoense
Mirassol
São Paulo
Atlético−MG
Bragantino
Internacional
Remo
Vasco
Corinthians
Athletico−PR
Cruzeiro
Botafogo
Fluminense
Coritiba
Bahia
Santos
Palmeiras
Flamengo
Times
12


θm
0
1
2
3
Corinthians
Internacional
Vasco
Coritiba
Mirassol
Vitória
Chapecoense
Santos
Bragantino
Remo
Bahia
São Paulo
Cruzeiro
Atlético−MG
Grêmio
Athletico−PR
Palmeiras
Fluminense
Botafogo
Flamengo
Times


Φ_v
0
1
2
3
Flamengo
Vitória
Palmeiras
Athletico−PR
Corinthians
São Paulo
Bragantino
Bahia
Internacional
Fluminense
Coritiba
Grêmio
Cruzeiro
Atlético−MG
Santos
Remo
Vasco
Mirassol
Botafogo
Chapecoense
Times


Φ_m
0
1
2
3
Palmeiras
Flamengo
Atlético−MG
Corinthians
Bragantino
Athletico−PR
São Paulo
Bahia
Coritiba
Fluminense
Grêmio
Internacional
Mirassol
Cruzeiro
Santos
Botafogo
Chapecoense
Vasco
Remo
Vitória
Times

Atividade Extra

Uma maneira de aprimorar as estimativas, seria por meio de uma taxa para simulação que não seja fixa, mas que se adapte conforme o campeonato evolui. Para isso, o mais usual é utilizar uma média móvel, com uma janela fixa (por exemplo, considerar a média das k últimas partidas), porém essa abordagem desconsidera os jogos mais antigos, em favor dos novos.

Uma maneira de mitigar isso, é por meio de Médias Móveis Exponencialmente Ponderadas (*Exponentially Weighted Moving Averages*, ou **EWMA**), que considera todas as partidas anteriores, com pesos que decaem de maneira exponencial. Assim, pondera-se o último jogo com mais importância, mas sem descartar os jogos passados.

```r
# O parâmetro alpha controla o "esquecimento" do modelo,
# alpha = 0.2 significa que o jogo atual tem 20% de peso
# na nova taxa, e o histórico 80%:
alpha <- 0.20

calcula_ewma <- function(vetor_gols) {
    accumulate(vetor_gols, ~ alpha * .y + (1 - alpha) * .x,
               .init = vetor_gols[1])[-1]
}

# Criamos o histórico de maneira linear, pois agora
# não há mais importância se o time atuou como mandante
# ou visitante em cada partida:
taxas_finais_ewma <- jogos_realizados %>%
    pivot_longer(
        cols = c(time_mandante, time_visitante),
        names_to = "mando",
        values_to = "time"
    ) %>%
    mutate(
        gols_feitos = ifelse(mando == "time_mandante",
                             gols_mandante, gols_visitante),
        gols_sofridos = ifelse(mando == "time_mandante",
                               gols_visitante, gols_mandante)
    ) %>%
    arrange(time, rodada) %>%
    group_by(time) %>%
    mutate(
        theta_hist = calcula_ewma(gols_feitos),
        phi_hist = calcula_ewma(gols_sofridos)
    ) %>%
    # Pegamos apenas o valor da taxa na última rodada jogada
    # pelo time para projetar o futuro:
    slice_tail(n = 1) %>%
    select(time, theta_ewma = theta_hist, phi_ewma = phi_hist) %>%
    ungroup()
```

```
taxas_finais_ewma %>%
    kable()
```

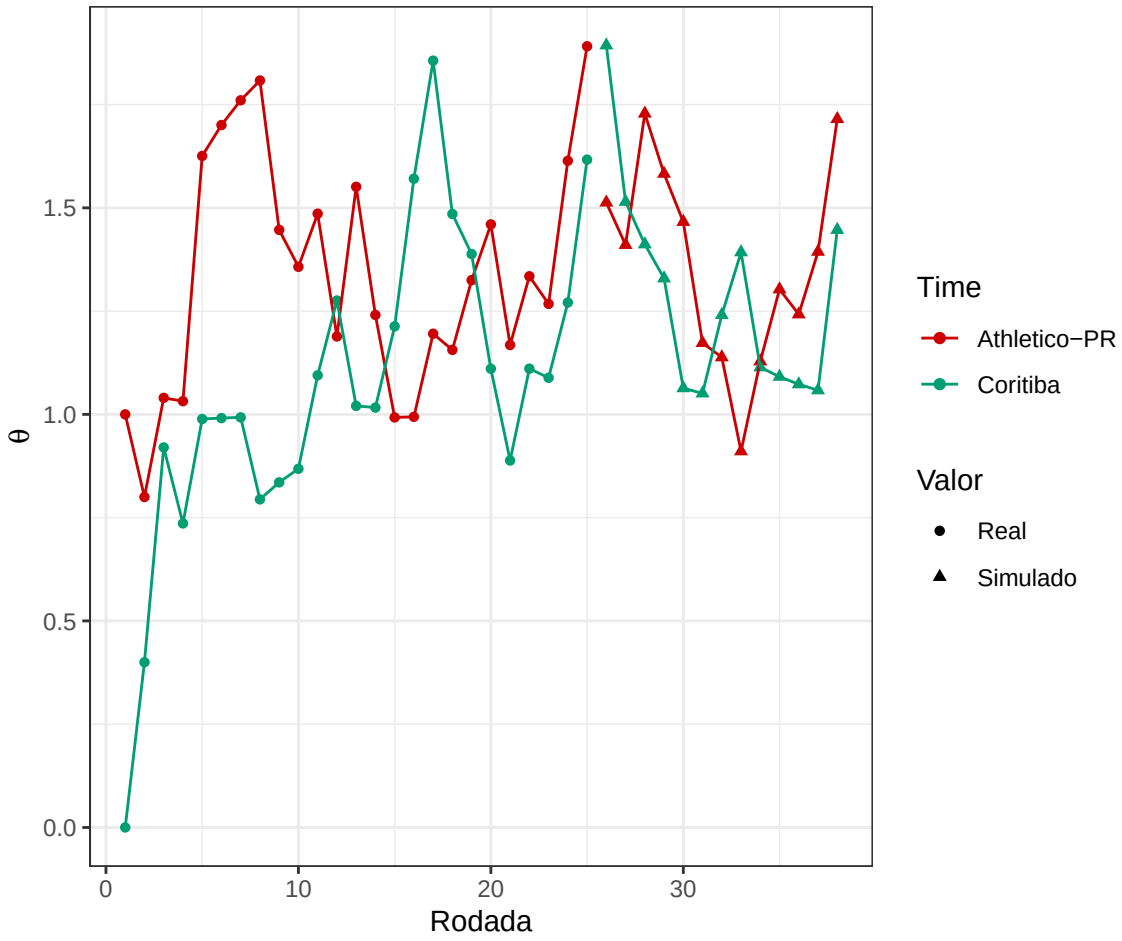
time	theta_ewma	phi_ewma
Athletico-PR	1.8912053	1.2570595
Atlético-MG	1.5353075	0.8283330
Bahia	1.7573311	1.2472952
Botafogo	1.0754578	1.7652459
Bragantino	1.0643867	1.0289341
Chapecoense	1.2415546	2.1138784
Corinthians	1.0209123	1.0885315
Coritiba	1.6166863	1.3993980
Cruzeiro	1.5614808	1.3671750
Flamengo	2.3272617	0.8066487
Fluminense	1.7950529	1.5081735
Grêmio	1.2273003	1.4424965
Internacional	0.9608387	1.3846542
Mirassol	1.1022806	1.7637946
Palmeiras	1.9450405	1.0278257
Remo	1.3750885	1.8520406
Santos	1.4190320	1.1115452
São Paulo	1.0553825	1.2771787
Vasco	1.0506111	1.8414949
Vitória	0.6641979	1.7067209

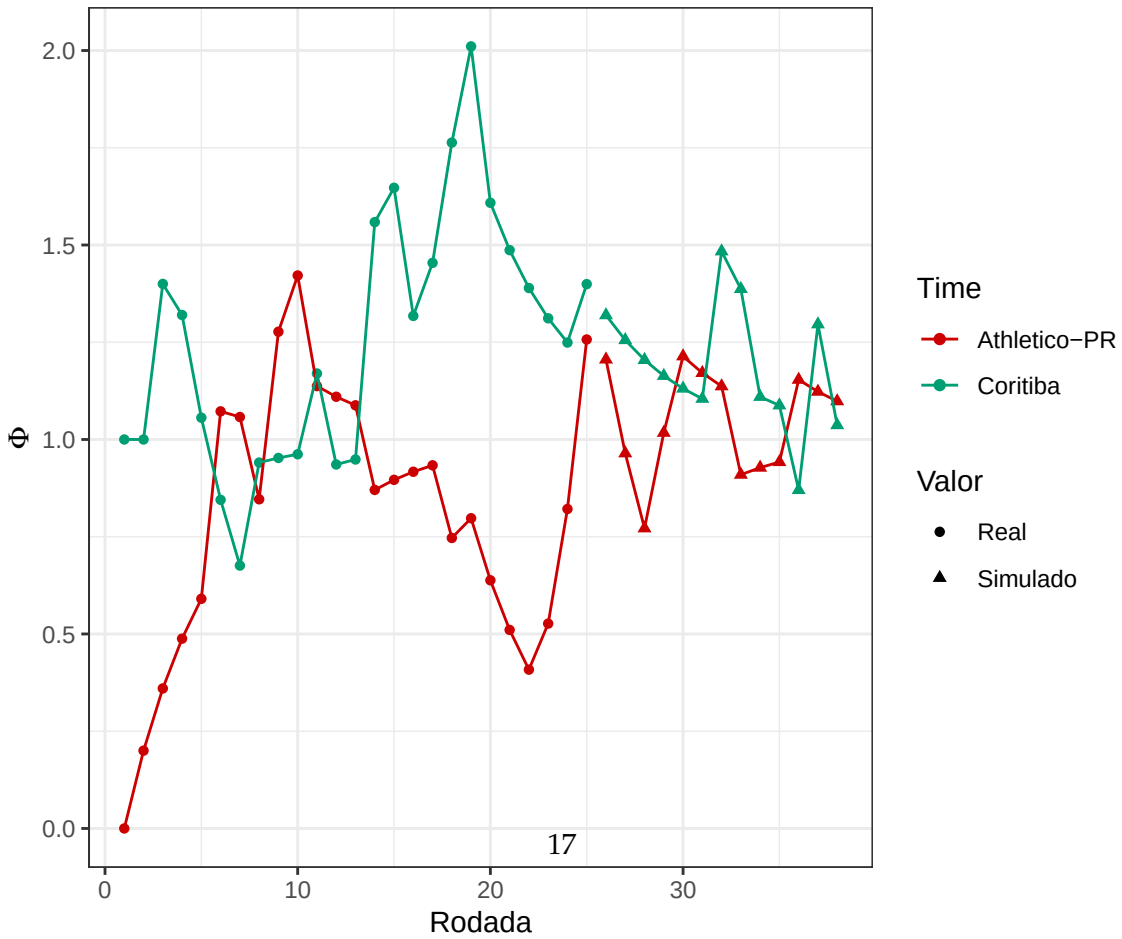
```
# Juntamos as taxas atualizadas com a tabela
# de jogos ainda não realizados:
jogos_futuros_ewma <- jogos_futuros %>%
    select(-c(theta_m, theta_v, phi_m, phi_v)) %>%
    left_join(
        taxas_finais_ewma %>% rename(time_mandante = time,
                                     theta_m = theta_ewma,
                                     phi_m = phi_ewma),
        by = "time_mandante"
    ) %>%
    left_join(
        taxas_finais_ewma %>% rename(time_visitante = time,
                                     theta_v = theta_ewma,
                                     phi_v = phi_ewma),
        by = "time_visitante"
    )

# Para simular, usamos um esquema semelhante ao anterior,
# porém com as taxas adaptativas:
simula_campeonato_ewma <- function(...) {
```

```
  sim_futuros <- jogos_futuros_ewma %>%
    mutate(
        gols_mandante = rpois(n(), (theta_m + phi_v)/2),
        gols_visitante = rpois(n(), (theta_v + phi_m)/2)
    )
  tabela_simulada <- calcula_tabela(sim_futuros)
  # Junta o passado real com o futuro simulado e aplica os critérios de desempate
  tabela_final <- bind_rows(tabela_base, tabela_simulada) %>%
      group_by(time) %>%
      summarise(across(everything(), sum), .groups = "drop") %>%
      arrange(desc(pontos), desc(vitorias), desc(saldo), desc(gols_pro)) %>%
      mutate(posicao = row_number())
  list(
      campeao = tabela_final$time[1],
      pontos_campeao = tabela_final$pontos[1],
      vice = tabela_final$time[2],
      pontos_vice = tabela_final$pontos[2],
      rebaixados = tabela_final$time[17:20],
      pontos_todos = tabela_final %>% select(time, pontos, posicao)
  )
}
```

Para ver como se comportariam as taxas adaptativas, rodemos uma simulação considerando os times paranaenses no campeonato:


θ
1.5
1.0
0.5
0.0
0
10
20
30
Rodada
Time
Athletico−PR
Coritiba
Valor
Real
Simulado


Φ
2.0
1.5
1.0
0.5
0.0
0
10
20
30
Rodada
Time
Athletico−PR
Coritiba
Valor
Real
Simulado

Com isso, realizemos a simulação uma quantidade grande de vezes, considerando agora a taxa adaptativa, e comparemos os resultados obtidos:

```
resultados_mc <- mclapply(1:n_simulacoes, simula_campeonato_ewma,
                          mc.cores = detectCores() - 2)
```

1. Qual a probabilidade de cada time ser campeão?

```
## Joining with `by = join_by(Time)`
```

Time	Freq_Ori	Freq_Adap
Flamengo	57.92%	58.0370%
Palmeiras	38.76%	37.8750%
Athletico-PR	2.85%	3.4030%
Fluminense	0.30%	0.3700%
Bahia	0.09%	0.1170%
Atlético-MG	0.02%	0.1120%
Cruzeiro	0.03%	0.0610%
Bragantino	0.02%	0.0130%
Coritiba	0.01%	0.0120%
Botafogo	0.00%	0.0000%
Chapecoense	0.00%	0.0000%
Corinthians	0.00%	0.0000%
Grêmio	0.00%	0.0000%
Internacional	0.00%	0.0000%
Mirassol	0.00%	0.0000%
Remo	0.00%	0.0000%
Santos	0.00%	0.0000%
São Paulo	0.00%	0.0000%
Vasco	0.00%	0.0000%
Vitória	0.00%	0.0000%

2. Qual a probabilidade de cada time ser rebaixado?

```
## Joining with `by = join_by(Time)`
```

Time	Freq_Ori	Freq_Adap
Chapecoense	24.8575%	24.72675%
Remo	18.5350%	17.95425%
Mirassol	15.1250%	14.14600%
Vasco	14.7675%	13.32900%
Internacional	12.2175%	10.95950%
Vitória	4.5425%	8.75850%
Grêmio	4.2050%	3.51550%
Botafogo	0.9200%	3.10675%
São Paulo	1.2225%	1.51850%
Santos	2.7100%	1.18950%
Corinthians	0.6900%	0.70475%
Bragantino	0.0525%	0.05175%
Coritiba	0.1000%	0.02975%
Atlético-MG	0.0300%	0.00400%
Cruzeiro	0.0200%	0.00375%
Bahia	0.0050%	0.00125%
Fluminense	0.0000%	0.00050%
Athletico-PR	0.0000%	0.00000%
Flamengo	0.0000%	0.00000%
Palmeiras	0.0000%	0.00000%

3. Qual a probabilidade do campeonato ser decidido pelos critérios de desempate?

```
## [1] "Original:"
```

```
## [1] "6.34%"
```

```
## [1] "Adaptativo:"
```

```
## [1] "6.53%"
```

4. Qual o valor esperado do número de pontos do campeão?

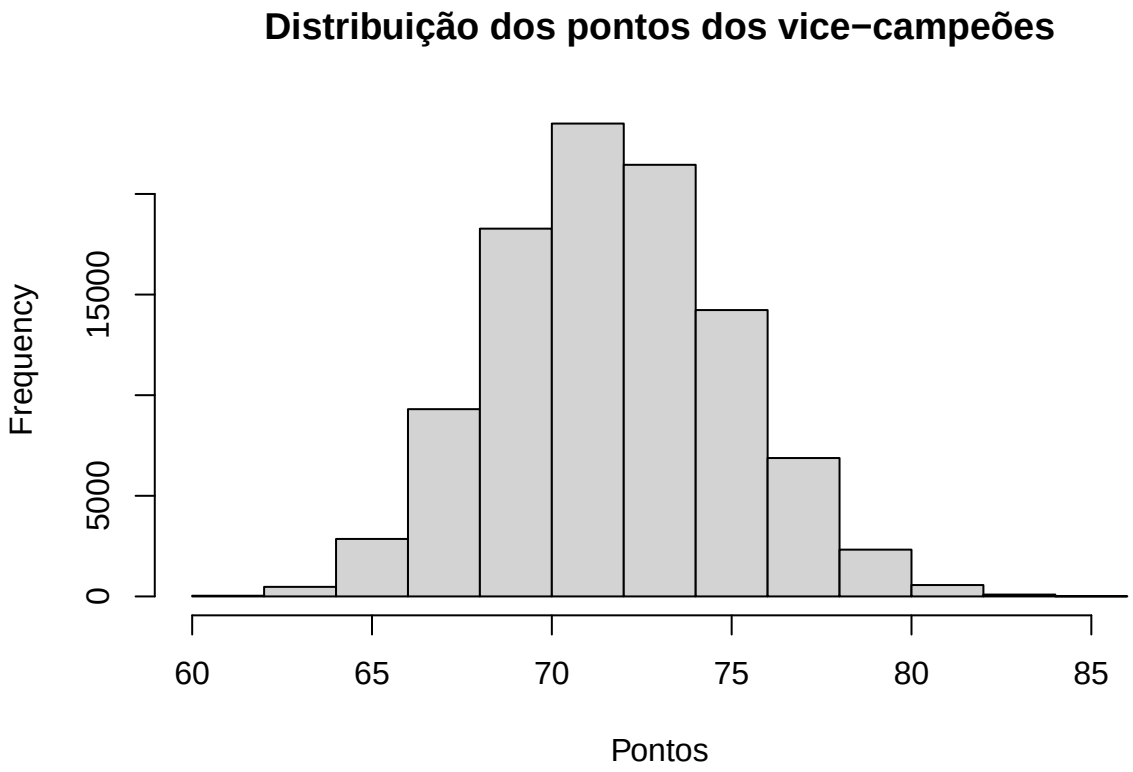
```
## [1] "Original:"
```

```
## [1] 76.974
```

```
## [1] "Adaptativo:"
```

```
## [1] 77.2113
```

5. Quantos pontos uma qualquer equipe precisa fazer para ter pelo menos 90% de chance de ser o campeão?



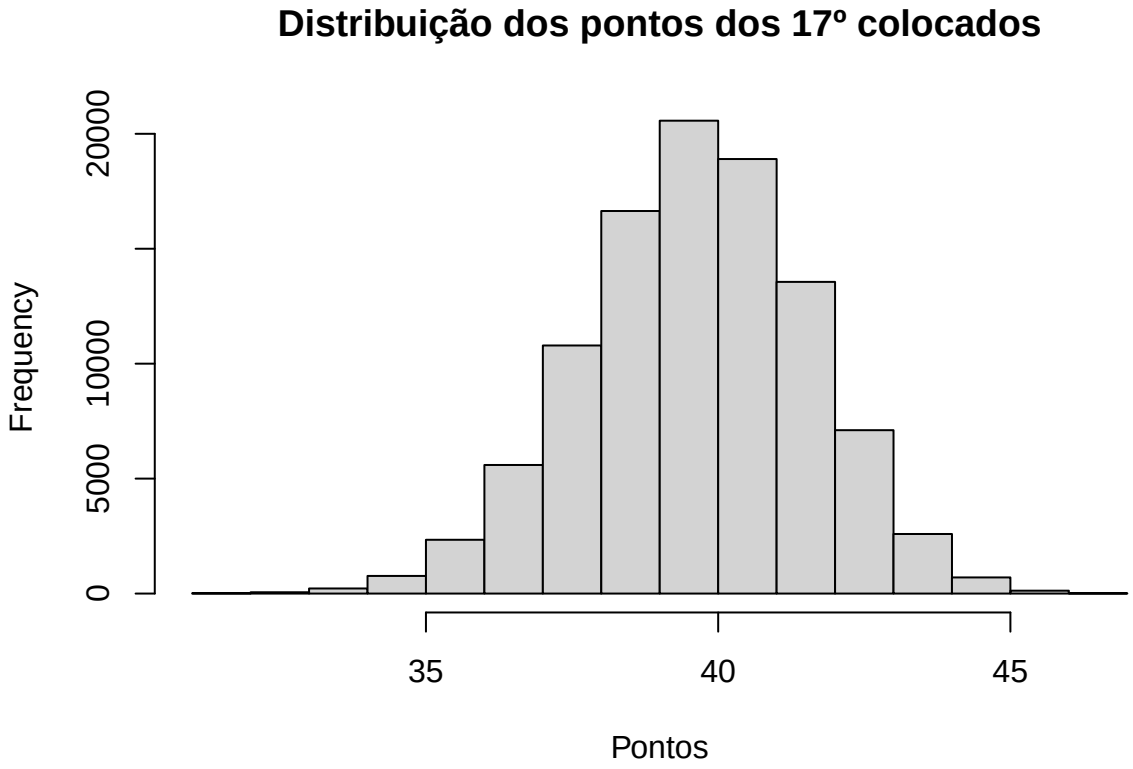

```
## [1] "Original:"

## 90%
##  76

## [1] "Adaptativo:"

## 90%
##  76
```

6. Quantos pontos são necessários que um time qualquer faça para ter pelo menos 95% de chance de não ser rebaixado?




```
## [1] "Original:"

## 95%
##  44

## [1] "Adaptativo:"

## 95%
##  43
```